Advanced Operating system

Cours: Process & Threads

B. Djellali & K.Belarbi

University of Science and Technology Mohamed Boudiaf Oran
Mathematics and Computer Science Faculty
Computer Science Department

9 mars 2026

Références bibliographiques

- A. Tanenbaum, *Modern Operating Systems*. Pearson, 2014.
- A. Tanenbaum, *Systèmes d'exploitation*. Dunod.
- S. Krakowiak, *Principes des systèmes d'exploitation*. Dunod.
- Autres supports : cours universitaires sur les processus, threads et synchronisation.

Concept of a programme

- Un programme est un ensemble d'instructions et de données stocké dans un fichier (exécutable, script, etc.).
- Il est passif : il réside sur un support de stockage (disque, SSD).
- Un même programme peut être exécuté plusieurs fois en parallèle.

Le programme définit ce qu'il faut faire, mais ne décrit pas à lui seul un contexte d'exécution particulier.

1. The process
   - Un process est une entité dynamique (*un program en cours d'exécution)*.
   - Il s'agit d'une entité active créée à un instant donné, qui disparaît en général au bout d'un temps fini.
   - *But* : A faire plusieurs activités "**en même temps**".
   - *Problème :* L'exécution d'un process est séquentiel, une seule instruction au plus est exécuté au nom du process.
   - Solution : Partager un (ou plusieurs) processor entre différents process.
   - Deux process associés au même program, sont deux séquences d'exécution ***process*** séparées.
   - Attention ! ! ! Ne pas confondre process et processor.

Programme vs process

Programme	Process
Fichier sur disque	Instance en mémoire
Statique	Dynamique
Peut engendrer plusieurs processus	A un état d'exécution
Aucune ressource associée	Possède des ressources (CPU, mémoire, fichiers)

Un même programme (par exemple gcc) peut donner naissance à plusieurs processus s'exécutant en parallèle.

Concept of process

- Un process est une instance d'un programme en cours d'exécution.
- Il possède :
  - un espace mémoire (code, données, pile, tas),
  - un contexte d'exécution (registres, compteur ordinal),
  - des ressources associées (fichiers ouverts, périphériques, etc.).

Le process est l'entité que le operating system crée, planifie et détruit pour exécuter des programmes.

Concept du process

The process

Un process est pourvu de :

- **PID P*rocess* ID*entifier*** : un numéro d'identification unique
- **un espace d'adressage :** un ensemble d'adresses mémoire allant de 0 à une limite donnée, dans lesquelles le processus peut lire et écrire. Il contient : *(i)*program exécutable ; *(ii)* ses données *(iii)* sa pile
- **Ensemble de registres**, parmi lesquels le compteur ordinal, le pointeur de pile, d'autres registres matériels
- les fichiers ouverts, les espaces mémoire alloués, les périphériques,
- les signaux à capter, à masquer, à ignorer, en attente et les actions associées,
- le process père, les process fils, le groupe, les variables d'environnement, les statistiques, les limites d'utilisation des ressources,

Concept du process

Le process

Exemple 1

- Faire travailler plusieurs users sur la même machine. **Chaque utilisateur a l'impression d'avoir la machine à lui tout seul**.
- Compiler tout en lisant son mail

Exemple 2

Sur le même ordinateur

- gcc prog1.c # compilation 1
- gcc prog2.c # compilation 2
- firefox # navigateur web
- eclipse # IDE java

Concept du process

Le process

Différence entre process et program

Un ***program*** **n'est pas un** ***process*** : un program est une entité passive, comme les contenus d'un fichier stocké sur un disc. Un program peut engendrer lors de son exécution plusieurs process, alors que le process est une entité active avec un compteur d'instructions spécifiant l'instruction suivante à exécuter et un ensembles de ressources associées.

$$\textbf{Process} = \textbf{instruction} + \textbf{ressources}$$

1. MultiProgrammation

- La **multi-programmation** donne l'illusion du parallélisme. À l'inverse, le temps partagé donne à chaque utilisateur L'impression d'être le seul utilisateur de la machine.
  La séparation du traitement pour simuler l'exécution en parallèle est appelée **multiprogrammation**.
- Si le modèle consiste à séparer le traitement entre différents processus, on parle de **multiprocessus** ou de **multitâche**.
  Si le partage se fait à l'intérieur d'un même processus, on parle de **multithread**.
- Le processeur n'en exécute qu'un seul. L'idée est de conserver plusieurs tâches dans la mémoire principale. Si une tâche est occupée par des opérations d'entrée/sortie, le processeur peut être affecté à une autre tâche.

2. multi-processeur

- Les machines multi-processeurs sont appelées machines parallèles et présentent la véritable notion de processus exécutés en parallèle.
- Organiser l'exécution des programmes sur différents processeurs (qui peuvent être identiques ou non). Ces systèmes peuvent être soit architecturés autour d'un processeur central qui coordonne les autres processeurs, soit avec des processeurs indépendants qui possèdent chacun leur système d'exploitation, ce qui leur vaut de communiquer entre eux par l'intermédiaire de protocoles.

3. Multi-Core MULTI-CORE : plusieurs coeurs physiques(unité de calcul) côte-à-côte sur la même puce (processeur)fonctionnant simultanément.
Le support (la connectique qui relie le processeur à la carte électronique) lui ne change pas. Certains éléments, comme par la RAM, peuvent être mis en commun.

Concept de processus

État du process

Quand un process s'exécute, il change d'état :

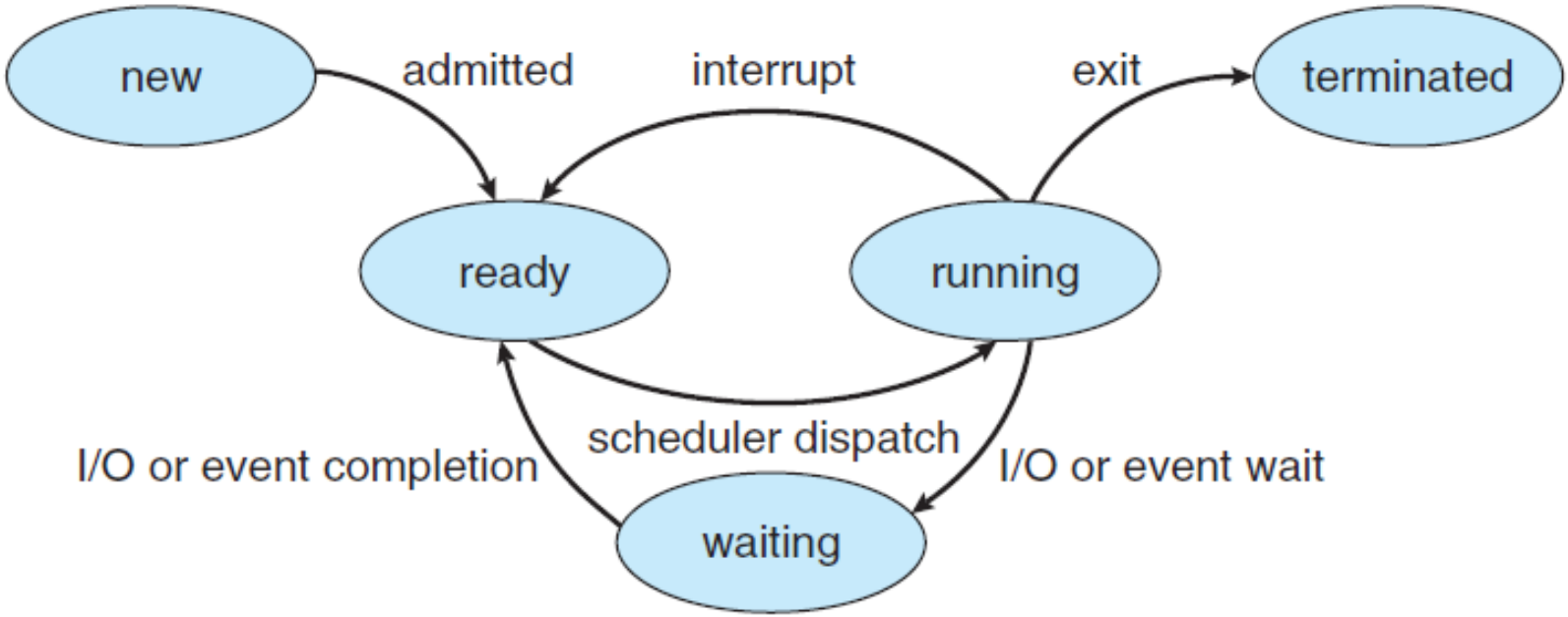
- **New** : le process est en cour de création ;
- **Ready** :le process attend d'être affecté à un processor ;
- **Running** : les instructions sont en cours d'exécution ;
- **Waiting** : le process attend qu'un événement se produise (comme l'achèvement d'un input output où la réception d'un signal) ;
- **Terminated** :le process a fini l'exécution.

Un seul process peut être en exécution sur n'importe quel processor à tout moment. Toutefois, plusieurs process peuvent être prêts et en attente

Concept du process

État du process

Figure – Diagramme d'état du process



Concept du process

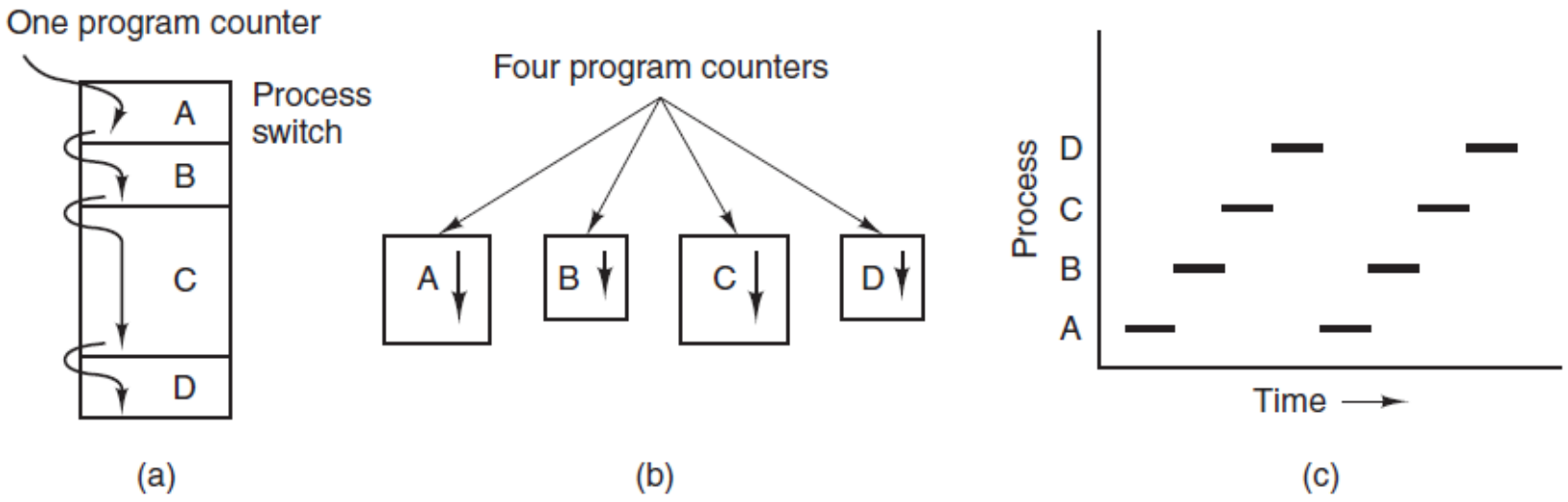
État du process

- Point de vue conceptuel : chaque process possède son virtual processor.
- Réalité : le processor bascule constamment d'un process à l'autre. Ce basculement rapide est appelé **multiprogrammation**.
- Lorsque le processor passe d'un process à un autre, la vitesse de traitement d'un process donné n'est pas uniforme et probablement non reproductible si le même process s'exécute une nouvelle fois.

Concept du process

État du process

Figure – (a) Multiprogrammation de quatre programs ; (b) Modèle conceptuel de quatre process séquentiels indépendants ; (c) un seul program est actif à un instant donné.



Concept du process

État du process

Exemple 2

Un informaticien porté sur la gastronomie, veut confectionner un gâteau d'anniversaire pour sa sœur. Il dispose d'une cuisine équipée, il dispose d'une recette et tous les ingrédients nécessaires : œufs, farine, sucre, extrait de vanille, etc.

- La recette représente the program (c'est à dire un algorithme exprimé en notation appropriée.)
- L'informaticien représente le processor (l'unité centrale)
- Les ingrédients sont les données d'entrée.
- Le process est l'activité qui consiste à confectionner le gâteau : lecture de la recette, mélange des ingrédients et la cuisson.

Maintenant notre informaticien se met à hurler, il vient d'être piqué par une guêpe.

- Son cerveau enregistre le point de la recette ou il en est arrivé*(l'état du process en cours est enregistré)*.
- Il cherche une brochure sur les premiers secours et commence à suivre les instructions.
- Ici, on voit le basculement du processor d'un process(confection du gâteau) vers un autre process de priorité supérieure *(administrer les premiers soins, un antidote)*
- chaque process déroule un program différent
- une fois le problème de la piqûre résolue, l'informaticien retourne à son gâteau au point ou il l'avait laissé.

Définition PCB :

- Chaque process est représenté par une structure de données au niveau du OS
- Un PCB (Process Control Block) est une structure de données du noyau qui contient toutes les informations nécessaires pour gérer un processus et pouvoir le suspendre/reprendre correctement.
- Lors d'un changement de contexte, le noyau sauvegarde l'état du processus courant dans son PCB, puis recharge l'état d'un autre processus à partir de son PCB.

PCB (Process Control Block)

le PCB contient plusieurs informations concernant un process spécifique : Ces informations sont dans une zone mémoire accessible uniquement par Kernel du OS.

Figure – PCB

Process Control Block (PCB)

- Chaque processus:
  - a un bloc de contrôle qui le décrit.
  - a un IDentifieur unique.
  - peut avoir des enfants ou un père.
  - a ses registres, sa mémoire et sa pile.
  - a une priorité.
- Les processus peuvent partager de la mémoire, des processus, des fichiers, des I/Os et autres.

Process ID (PID)
Pointer to parent process
Pointers to child processes
Process state
Program Counter
Registers
Memory pointers
Priority information
Accounting information
Pointers to shared ressources

PCB Typique

Pour obtenir des informations sur les processes, il est donc nécessaire de passer par des appels systèmes : **getpid, getppid, getuid, geteuid, setuid**. . .

- **Pointeur** - Il s'agit d'un pointeur de pile qui doit être sauvegardé lorsque le process passe d'un état à un autre pour conserver la position actuelle du process.
- **Process state** - Il stocke l'état respectif du process.
- **Process IDentifier (PID)** - Chaque process se voit attribuer un identifiant unique (à un instant t, mais réutilisable) connu sous le nom *Processus ID* ou **PID** qui stocke l'identifiant du processus.
- **Program counter** - Il stocke le compteur qui contient l'adresse de l'instruction suivante qui doit être exécutée pour le processus.
- **Registers** - Il s'agit des registers du CPU qui comprennent : l'accumulateur, la base, les registres et les registres d'usage général.
- **Process scheduling information :** information concernant la priorité du process.
- **Memory information :** valeurs des registres base et limite, des tables de pages ou des tables de segments.
- **Informations sur l'état des E/S :** liste des périphériques E/S allouées à ce processus, une liste des fichiers ouverts, etc.

Concept du process

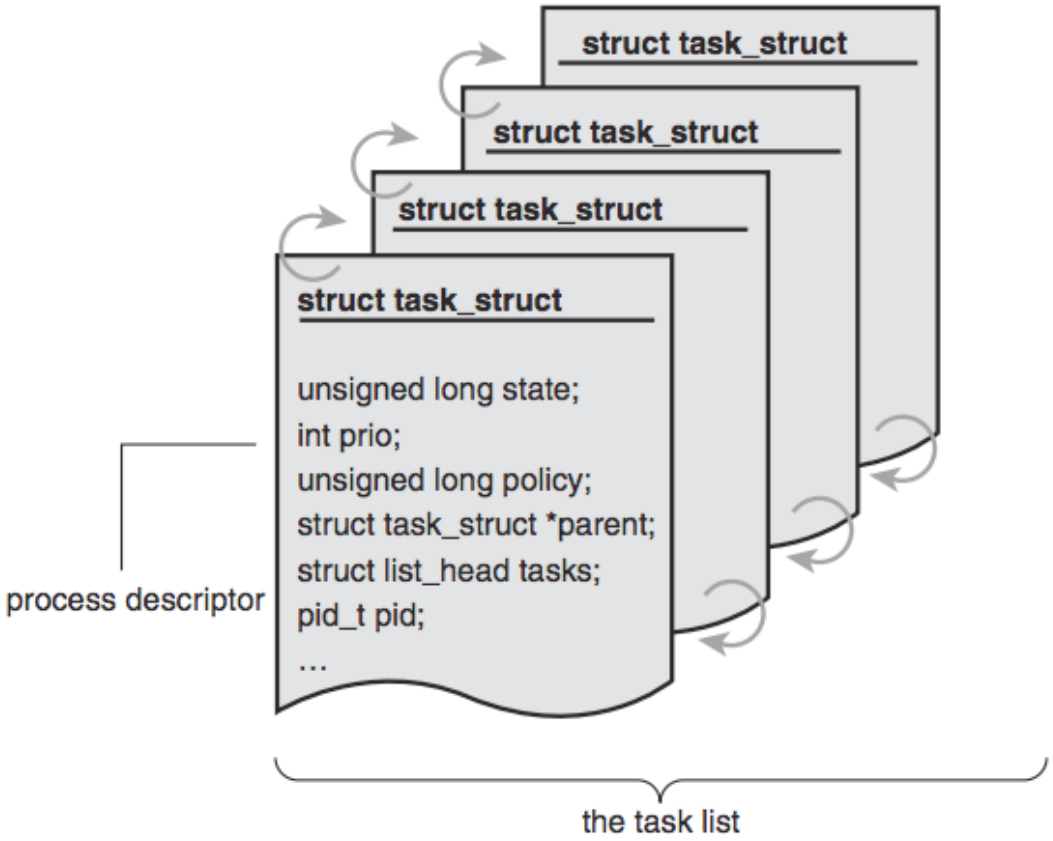
Linux/UNIX : Lister des informations sur les process

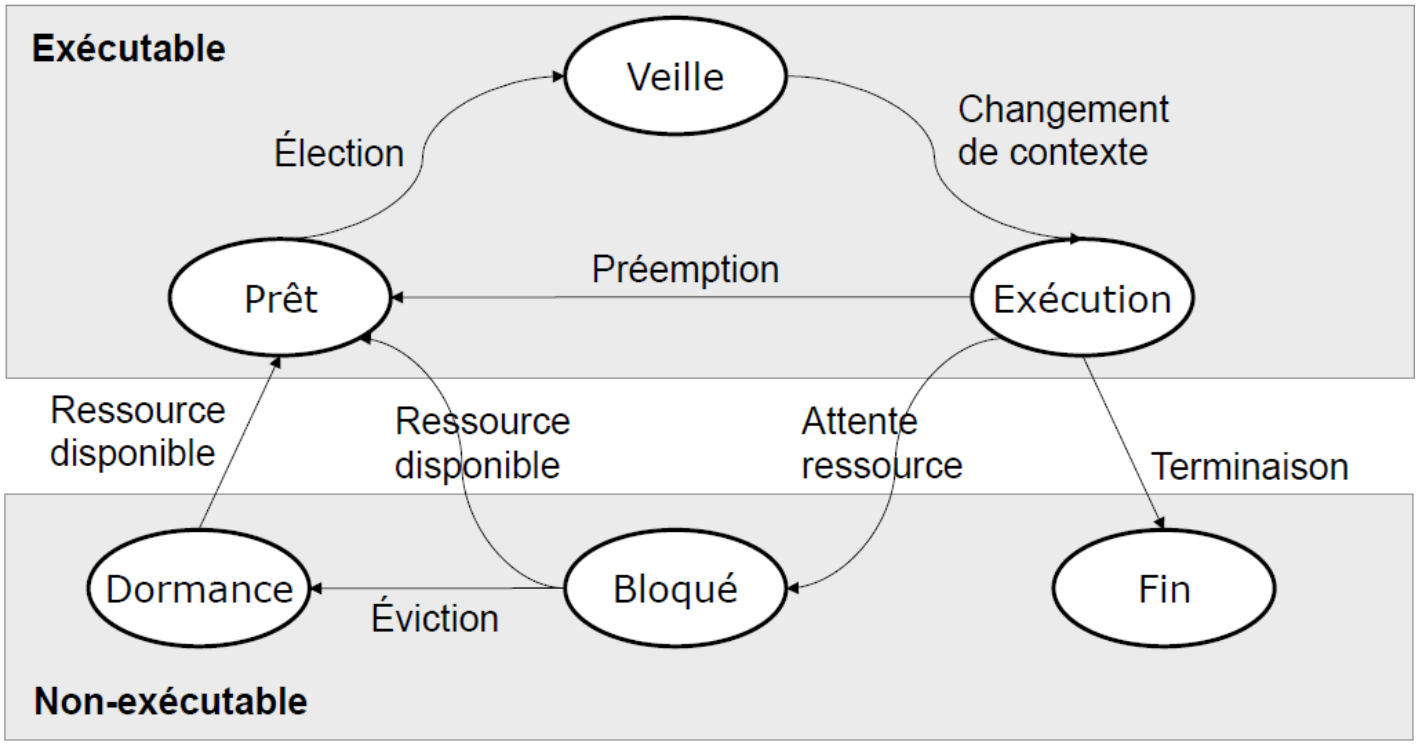
```
crw-------  1 root root     10, 137 29 janv. 16:40 vhci
crw-rw----  1 root kvm      10, 238 29 janv. 16:40 vhost-net
crw-rw----  1 root kvm      10, 241 29 janv. 16:40 vhost-vsock
crw-------  1 root root     10, 123 29 janv. 16:40 vmci
crw-rw-rw-  1 root root     10, 122 29 janv. 16:40 vsock
crw-rw-rw-  1 root root      1,   5 29 janv. 16:40 zero

┌──(kali㉿kali)-[~]
└─$ ps -af
UID          PID    PPID  C STIME TTY          TIME CMD
kali        2548    1328  0 16:43 pts/0    00:00:00 ps -af

┌──(kali㉿kali)-[~]
└─$ ps -af
UID          PID    PPID  C STIME TTY          TIME CMD
kali        2812    2612  0 16:44 pts/1    00:00:00 nano
kali        2920    1328  0 16:44 pts/0    00:00:00 ps -af

┌──(kali㉿kali)-[~]
└─$
```

```
┌──(kali㉿kali)-[~]
└─$ pwd
/home/kali

┌──(kali㉿kali)-[~]
└─$ nano &
[1] 2812

[1]  + suspended (tty output)  nano
┌──(kali㉿kali)-[~]
└─$
```

- The process ID ou **PID** : un nombre d'identification unique utilisé à faire référence au process
- The parent process ID ou **PPID** : un nombre d'identification du process (PID) qui a lancé ce process
- Terminal ou TTY terminal auquel ce process est connecté

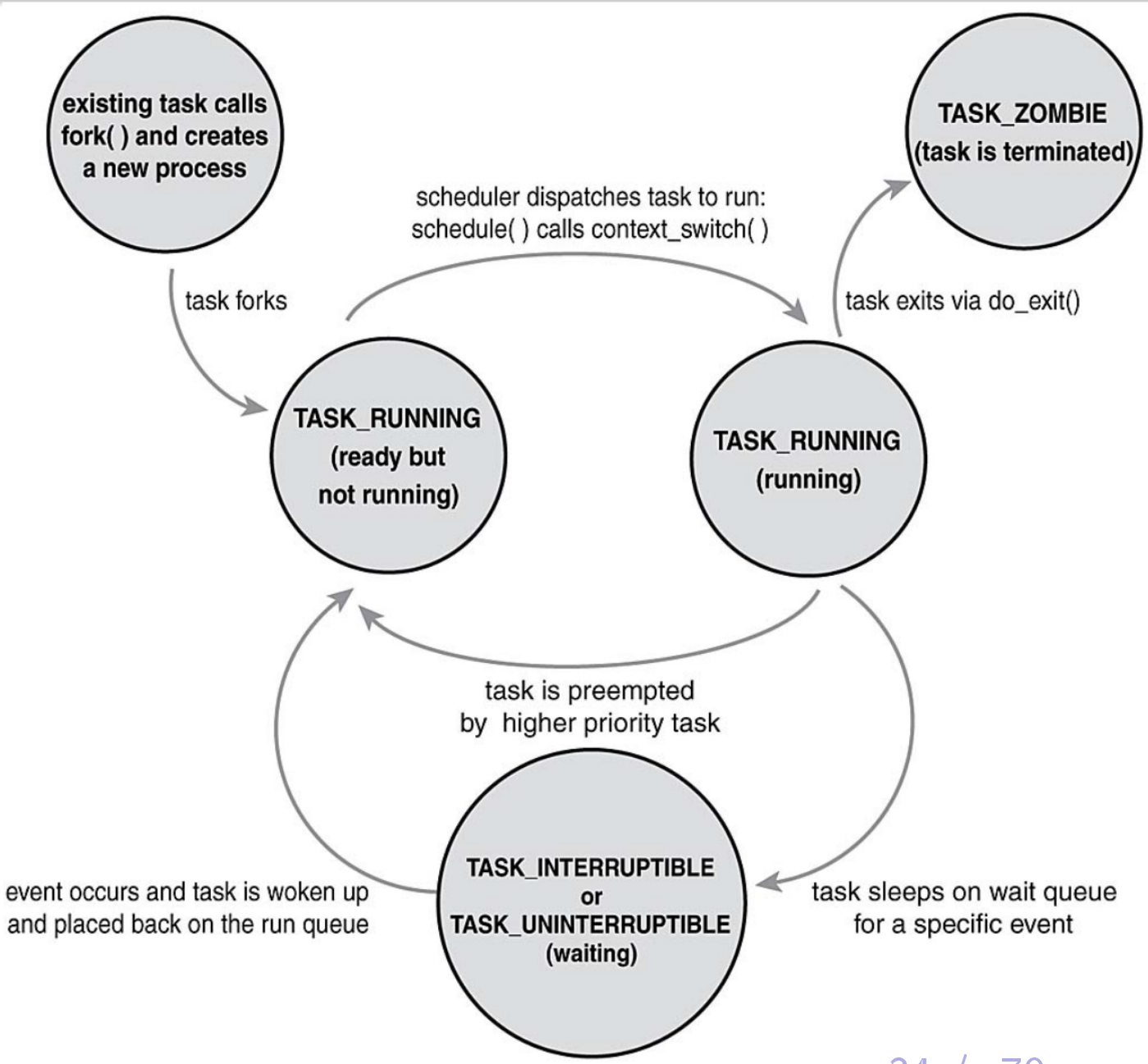
Dans le cas de Linux, le PCB est une structure task_struct et la table des processus est une liste doublement chainée.




Exécutable
Veille
Élection
Changement
de contexte
Prêt
Préemption
Exécution
Ressource
disponible
Ressource
disponible
Attente
ressource
Terminaison
Dormance
Éviction
Bloqué
Fin
Non-exécutable

Concept du process

État d'un process *sous Linux*



Concept du process

État d'un process *sous Linux*

- R à Task_Running,
- S (sleeping) à Task_Interruptible : peuvent être interrompus par des signaux et des processus d'attente ,
- D à Task_Uninterruptible : attendent directement dans des conditions matérielles et ne peuvent être interrompus par aucun événement / signal.,
- T (being traced or stopped) à Task_Stopped,
- Z à Task_Zombie : un processus est mort, il a été arrêté, mais il a toujours une entrée dans la table de processus.

Executer la commande **top ps** Linux (utile en **SSH**)

- **Lire les informations et comprendre les détails des colonnes**

- Utiliser **la touche v**(top) pour voir la hiérarchie des process (process parents et enfants)

Fichier Actions Éditer Vue Aide

```
top - 07:49:04 up 5 min,  2 users,  load average: 0,22, 0,23, 0,11
Tâches: 169 total,   1 en cours, 168 en veille,   0 arrêté,   0 zombie
%Cpu(s):  1,4 ut,  1,9 sy,  0,0 ni, 96,6 id,  0,0 wa,  0,0 hi,  0,2 si,
MiB Mem :   1974,1 total,    999,3 libr,    635,0 util,    484,6 tamp/c
MiB Éch :   1024,0 total,   1024,0 libr,      0,0 util.   1339,1 dispo
```

PID	UTIL.	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TEMPS+
714	root	20	0	387416	99300	56824	S	1,3	4,9	0:09.25
1041	kali	20	0	984448	127040	84300	S	0,7	6,3	0:02.14
39	root	20	0	0	0	0	I	0,3	0,0	0:00.02
1102	kali	20	0	287184	48400	20252	S	0,3	2,4	0:00.93
1249	kali	20	0	336296	24248	17500	S	0,3	1,2	0:00.10
2625	kali	20	0	463864	106536	89908	S	0,3	5,3	0:01.06
3569	root	20	0	0	0	0	I	0,3	0,0	0:00.02
1	root	20	0	22972	13608	10116	S	0,0	0,7	0:00.96
2	root	20	0	0	0	0	S	0,0	0,0	0:00.01
3	root	20	0	0	0	0	S	0,0	0,0	0:00.00
4	root	0	-20	0	0	0	I	0,0	0,0	0:00.00
5	root	0	-20	0	0	0	I	0,0	0,0	0:00.00
6	root	0	-20	0	0	0	I	0,0	0,0	0:00.00

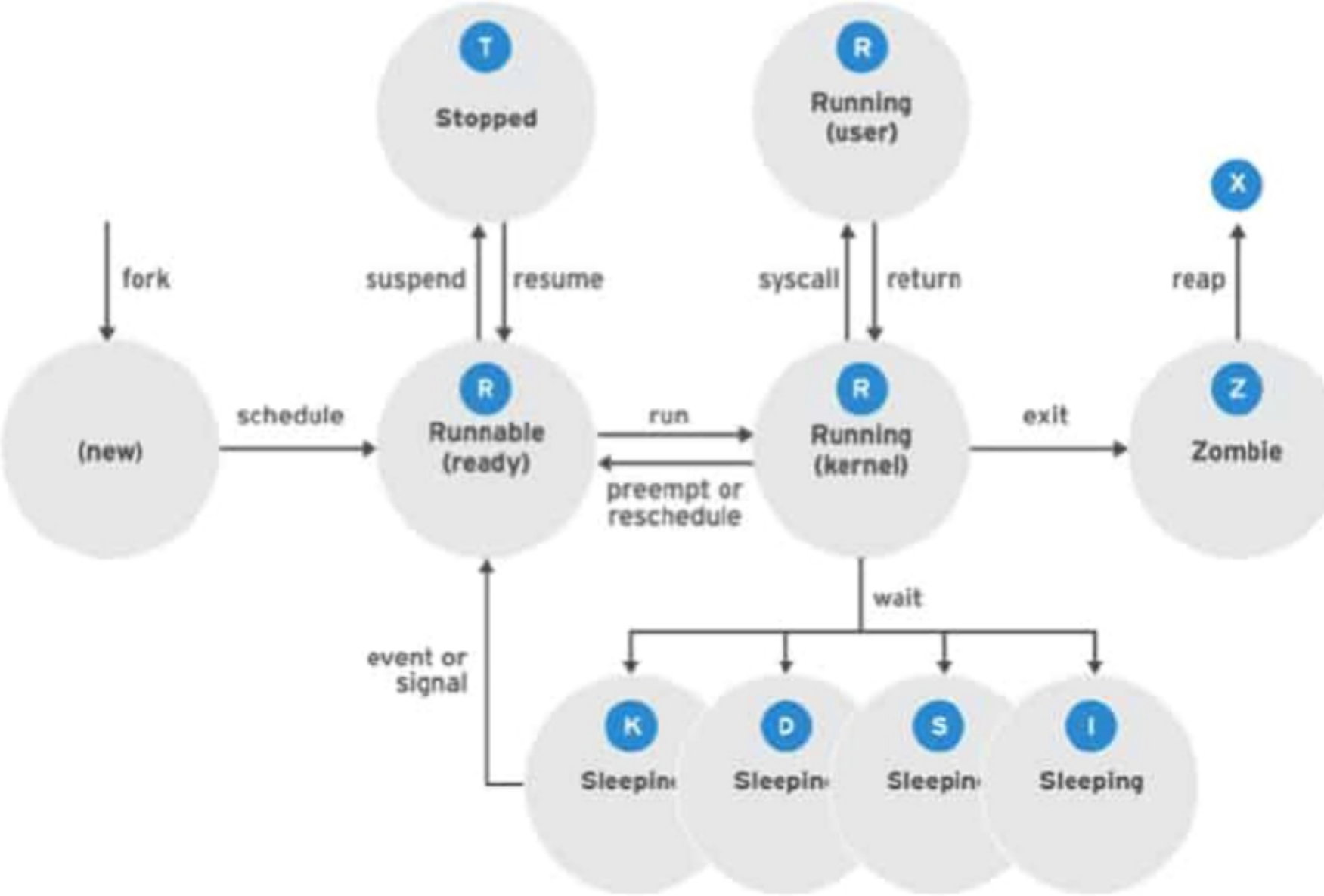
les états peuvent avoir des sous-états identifiés par des drapeaux (flags).

Nom	Flag	Nom d'état et description définis par le noyau
Running	R	TASK_RUNNING : Le process est soit en cours d'exécution sur une CPU, soit en attente d'exécution dans la file d'attente. Le process peut exécuter des routines utilisateur ou des routines du kernel (appels système).
Sleeping	S	TASK_INTERRUPTIBLE : Le process attend une condition : une demande matérielle, un accès des ressources système ou un signal. Lorsqu'un événement ou un signal satisfait à la condition, le process passe a TASK_RUNNING.
Sleeping	D	TASK_UNINTERRUPTIBLE : Ce process dort également, mais contrairement à *S State*, ne répond pas aux signaux. Utilisé uniquement lorsque l'interruption du process peut provoquer un état de périphérique imprévisible.
Sleeping	K	TASK_KILLABLE : Cet état est similaire à state D (TASK_UNINTERRUPTIBLE) , mais il réveille le process si un signal fatal (c'est-à-dire un signal qui tuerait le process) est reçu.
Sleeping	I	TASK_REPORT_IDLE : Un sous-ensemble state D. Kernel ne compte pas ces process lors du calcul de la charge moyenne . Utilisé pour les threads du Kernel. Le flag Task_UnInterruptible et Task_NoLoad sont définis. Semblable à Task_Killable, également un sous-ensemble state D. Il accepte des signaux fatals.

Nom	Flag	Nom d'état et description définis par le noyau
Stoped	T	TASK_STOPPED : Le process a été arrêté (suspendu), généralement en étant signalé par un utilisateur ou un autre process. Le process peut être poursuivi (repris) par un autre signal pour revenir Running.
Stoped	T	TASK_TRACED : Un process en cours de débogage est également temporairement arrêté et partage le même flag state T.
Zombie	Z	EXIT_ZOMBIE : A child process signale à son parent qu'il se termine. Toutes les ressources, à l'exception de l'identité du process (PID), sont libérées.
Zombie	X	EXIT_DEAD : Lorsque le parent nettoie la structure du child process restante, le process est maintenant libéré complètement. Pour effacer le process enfant terminé de la table des process, le process parent doit lire la valeur de sortie du child process à l'aide des appels système wait() ou waitpid().

Concept du process

État d'un process *-sous Linux-*



Si la machine possède un processeur à plusieurs cores alors ces derniers s'affichent en liste.

```
top - 14:03:02 up 1037 days,  8:45,  3 users,  load average: 0,12, 0,21, 0,27
Tasks: 292 total,   1 running, 290 sleeping,   0 stopped,   1 zombie
%Cpu0  :  5,5 us,  0,8 sy,  0,0 ni, 88,1 id,  5,4 wa,  0,0 hi,  0,1 si,  0,0 st
%Cpu1  :  5,7 us,  0,9 sy,  0,0 ni, 88,1 id,  5,3 wa,  0,0 hi,  0,1 si,  0,0 st
%Cpu2  :  5,7 us,  0,8 sy,  0,0 ni, 89,3 id,  4,2 wa,  0,0 hi,  0,1 si,  0,0 st
%Cpu3  :  5,6 us,  0,8 sy,  0,0 ni, 89,3 id,  4,3 wa,  0,0 hi,  0,1 si,  0,0 st
%Cpu4  :  3,4 us,  0,6 sy,  0,0 ni, 84,3 id, 11,7 wa,  0,0 hi,  0,0 si,  0,0 st
%Cpu5  :  2,9 us,  0,5 sy,  0,0 ni, 88,8 id,  7,7 wa,  0,0 hi,  0,0 si,  0,0 st
%Cpu6  :  2,6 us,  0,4 sy,  0,0 ni, 91,0 id,  5,8 wa,  0,0 hi,  0,0 si,  0,0 st
%Cpu7  :  2,9 us,  0,5 sy,  0,0 ni, 88,9 id,  7,7 wa,  0,0 hi,  0,0 si,  0,0 st
KiB Mem:  32960488 total, 32256708 used,   703780 free,   261996 buffers
KiB Swap:  1046520 total,   680672 used,   365848 free,  6160872 cached
```

*site malekal.com

Il existe quatre événements provoquant la création d'un process

- Initialisation du system
- Exécution d'un appel system de création d'un process par un processus en cours d'exécution par **fork()**
- Demande d'un utilisateur sollicitant la création d'un nouveau process
- Lancement d'un travail de traitement par lot (mainframes)

Les différentes valeurs renvoyées par fork() :

- **Valeur négative** : La création d'un processus enfant a échoué.
- **Zéro** : Renvoyé au processus fils nouvellement créé.
- **Valeur positive** : Renvoyé au parent ou à l'appelant. La valeur contient l'identifiant du processus enfant nouvellement créé.

Au démarrage un seul processus créé : **init** qui a pour descendants tous les processus qui s'exécuteront sur le système.
Sous UNIX un seul appel système pour créer de nouveau processus

int fork () ; *#include <unistd.h>*

int waitpid (int pid, int *status, int option) ; *#include <sys/wait.h>*

- pid numéro a attendre (-1 n'importe quel descendant)
- status valeur de sortie ou signal
- option en général 0 (voir man)
- Fournit le PID du process qui a changé,-1 en cas d'erreur

Plus généralement il se crée une hiérarchie entre les processus avec des connexions entre chaque processus et ses descendants

Un process s'arrête pour divers raison

- Arrêt normal*(volontaire)*
- Arrêt pour raison *(volontaire)*
- Arrêt pour erreur fatale*(involontaire)*
- le process est arrêté par un autre process(involontaire)
- arrêt volontaire : La commande exit()
- arrêt involontaire :Une commande spéciale est utilisée à cette fin : **KILL *name_process***

La terminaison d'un processus entraîne :

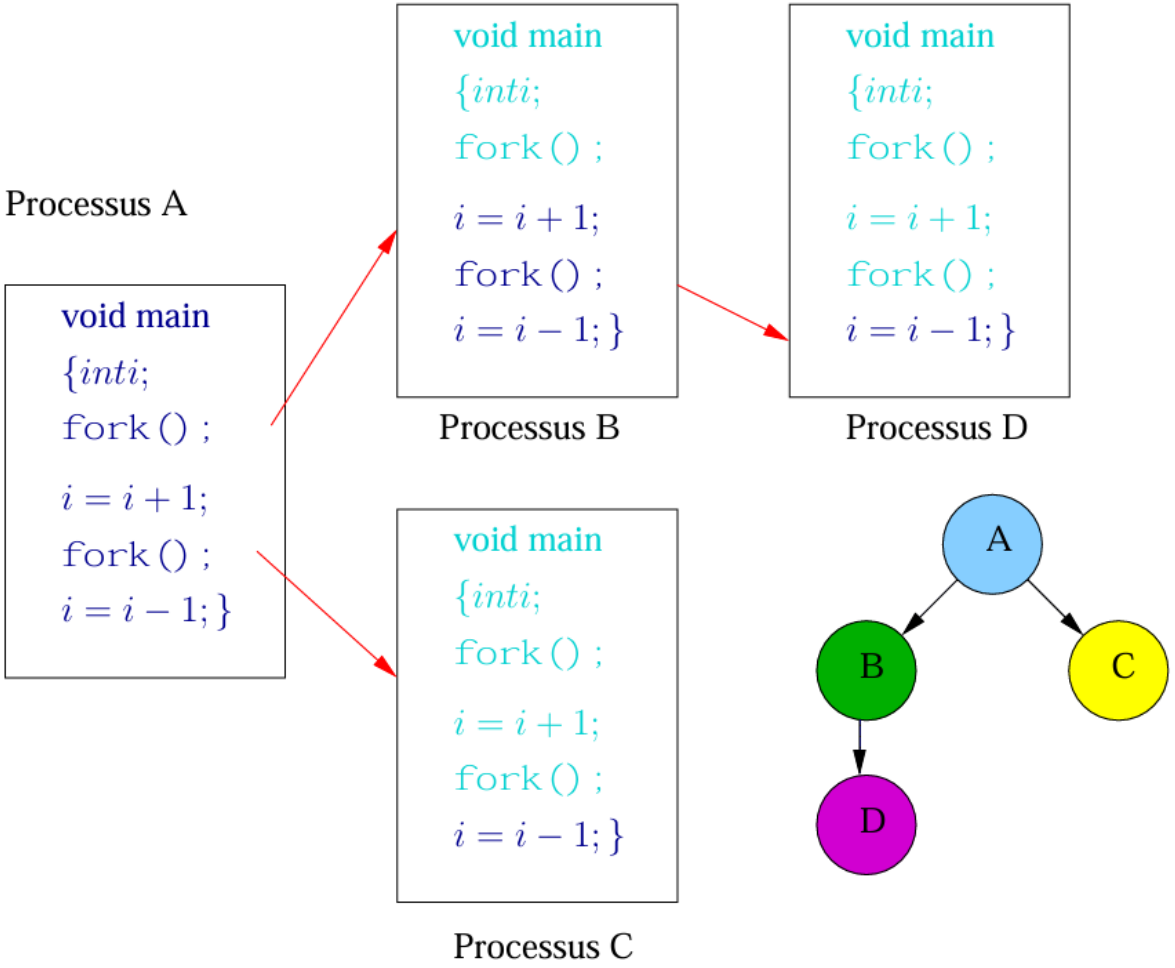
- la destruction de sa descendance
- La libération de l'entrée correspondante dans la table des processus
- La libération des ressources occupées par le processus.

int execve(const char *pathname, char *const argv [], char *const envp []) ;

#include <unistd.h>

execve() exécute le programme désigné par pathname. Cette méthode le programme actuellement exécuté par le processus appelant est remplacé par un nouveau programme, avec une nouvelle initialisation de la fonction la pile, le tas et les segments de données (initialisés et non initialisés) nouvellement initialisés.
pathname : chemin de l'exécutable : doit être soit un exécutable binaire, soit un script commençant par une ligne de la forme :
!interpreter [optional-arg]
argv : tableau de paramètres transmis a main
envp : tableau de variables d'environnement

- Un proces peut créer un ou plusieurs process qui, à leur tour, peuvent en créer d'autres.
- Les process parents et enfants continuent d'être associés (d'une certaines manière)
- Un processus n'a qu'un seul parent et peut avoir zéro, un, deux enfants, ou plus.
- **Windows** ne gère pas de relation hiérarchique entre processus
  - Tous les process sont égaux
  - Si ce parent se termine, il n'y a pas d'actualisation de cette information. Un processus peut pointer vers un parent inexistant. Cela ne cause aucun problème puisque rien ne dépend de cette information de parenté.
- **UNIX**, un process et l'ensemble de ses enfants et de ces descendants forment un groupe de process.
- il existe une arborescence des processus que l'on peut afficher avec **la commande pstree** (Linux).


Processus A
void main
{inti;
fork () ;
i = i + 1;
fork () ;
i = i − 1; }
void main
{inti;
fork () ;
i = i + 1;
fork () ;
i = i − 1; }
Processus B
void main
{inti;
fork () ;
i = i + 1;
fork () ;
i = i − 1; }
Processus D
void main
{inti;
fork () ;
i = i + 1;
fork () ;
i = i − 1; }
Processus C
A
B
C
D

- BIOS (Basic Input/Output System dans la carte mère) exécute des opérations de vérification de l'intégrité du système, charge et lance l'exécution du programme d'amorçage MBR (Master Boot Record).
- MBR charge une partie du système d'exploitation puis lance son exécution : le process 0 est créé
- Le process 0 réalise d'autres initialisations (ex. le système de fichier) puis crée deux processus : **init** (PID 1) et **démon** des pages (PID 2). Ensuite, d'autres processus sont crées à partir du processus init.
- le noyau Linux (kernel) exécute le processus init et donc ce dernier n'a pas de processus parent.
- Le processus initial fonctionne comme un parent adoptif pour tous *les processus orphelins*

- **getpid()** permet de récupérer le PID du processus.
- **getppid()** permet de récupérer le PID de son processus père.
- **fork()** La création de processus est réalisée par duplication de l'espace d'adressage et de certaines tables du processus créateur .
- La duplication facilite la création et le partage de ressources. Le fils hérite les résultats des traitements déjà réalisés par le père.
- **exec** permet de remplacer son code exécutable par un autre .
- **kill** permet de stopper/repartir/détruire (appel système kill)
- **wait** permet d'attendre la terminaison de ses fils mais ne peut pas les renier.
- Les appels système ***wait(& status)*** et ***waitpid(pid, & status, option)*** permettent au processus père de récupérer, dans status, cet état de terminaison.

Exemple 1 : Création d'un processus fils (chaque processus a son espace d'adressage (privé))

```
1  // programme tfork.c : appel système fork()
2  #include <sys/types.h>  /* typedef pid_t */
3  #include <unistd.h>                  /* fork() */
4  #include <stdio.h>                   /* pour perror, printf */
5  int a=20;
6  int main( ) {
7     pid_t x;
8     // création d'un fils
9     switch (x = fork()) {
10    case -1: /* le fork a échoué */
11       perror("le fork a échoué !");
12       break;
13    case 0:   /* seul le processus fils exécute  ce « case »*/
14       printf("ici processus fils, le PID %d.\n ", getpid());
15       a += 10;
16       break;
17    default:  /* seul le processus père exécute ce « case »*/
18       printf("ici processus père, le PID %d.\n", getpid());
19       a += 100;
20    }
21    // les deux processus exécutent ce qui suit :
22    printf("Fin du Process %d. avec a = %d\n", getpid(), a);
23    return 0;
24 }
```

```
ici processus père, le PID 12339.
ici processus fils, le PID 12340.
Fin du Process 12340 avec a = 30.
Fin du Process 12339 avec a = 120.
```

Exemple 2 : Création de plusieurs processus
- strace permet de construire l'arbre des processus créés.
- Orphelins adoptés par le processus init de pid 1.

```
1  // programme tree.c : appel système fork()
2  #include <sys/types.h>
3  #include <unistd.h>
4  #include <stdio.h>
5  int main( ) {
6      pid_t p;
7      int i, n=5;
8      for (i=1; i<n; i++) {
9          p = fork();
10         if (p > 0)
11             break ;
12         sleep(1);
13         printf(" Processus %d  de père %d. \n",  getpid(), getppid());
14 }
15     sleep(2);
16     return 0;
17 }
```

$strace -f -e trace=clone -o stree.txt ./tree
$cat stree.txt

Un process peut attendre ou vérifier la terminaison d'un de ses fils
– pid_t wait (int * pstatus) ; // attendre un fils
– pid_t waitpid(int pid, int*pstatus, int options) ; //attendre le fils spécifié.
wait(&status) et waitpid(pid, &status, options) retournent :
– le PID du fils qui s'est terminé,
– -1 en cas d'erreur (le processus n'a pas de fils), et
– dans status des informations sur l'état de terminaison. Ces informations peuvent être récupérées au moyen de **macros** telles que :

- **WIFEXITED(*status*)** : fin normale avec exit,
- **WIFSIGNALED(*status*)** : tué par un signal,
- **WIFSTOPPED(*status*)** : stoppé temporairement,
- **WEXITSTATUS(*status*)** : valeur de retour du processus fils (exit(valeur)).

waitpid(pid, &status, WNOHANG) vérifie seulement la terminaison pas d'attente. Il retourne 0 si l'exécution du processus n'est pas terminée.

Exemple 3 : État de terminaison d'un processus (rôles de wait et exit)

```
// programme deuxfils.c
#include <sys/wait.h>
#include <stdio.h>
#include <unistd.h>
void fils(int i);
int main() {
    int status;
    if (fork()==0) { // premier fils
            fils(1);
    } else if (fork()==0) { // second  fils
                    fils(2);
    } else  { if (wait(&status) > 0)
                        printf("fin du fils%d\n", WEXITSTATUS(status));
                    if (wait(&status) > 0)
                        printf("fin du fils%d\n", WEXITSTATUS(status));
        }
    return 0;
}
void fils(int i) {
    sleep(2);
    _exit(i);
}
```

Le passage de l'exécution d'un processus à un autre est appelé commutation de contexte.
Ces commutations peuvent être initiées par les programmes eux-mêmes (**multitâche coopératif**) ou par le système d'exploitation (**multitâche préemptif**).
Le multitâche préemptif est assuré par l'ordonnanceur (scheduler).
L'ordonnancement est le mécanisme par lequel un ordonnanceur, ou un kernel OS, choisit le process auquel on doit attribuer un processor.
Parmi les techniques d'ordonnancement les plus courantes :

1. Ordonnancement circulaire (tourniquet, round-robin) ;
2. Ordonnancement par priorité ;
3. Ordonnancement par priorité avec files multiples.

Les critères d'attribution d'un processor sont très différents selon qu'il s'agit de gérer un grand nombre de terminaux connectés à un ordinateur de temps partagé (time sharing) ou qu'il s'agit de gérer un process industriel en temps réel.

- Attendre le blocage ou la terminaison du processus qui s'exécute
- Provoquer le changement
  - Ranger les registres physiques dans la table des processus
  - Choisir un nouveau processus à exécuter
  - Charger les registres physiques à partir de la table des processus
- Comment forcer l'exécution de ces trois étapes ?
- Utilisation d'une horloge externe et du mécanisme des interruptions

Un signal est un moyen de communication indiquant une action à entreprendre à partir de conventions préétablies.

- Chaque signal a une signification particulière qui détermine le comportement du processus.
- Il n'y a pas de communication de données.
- Un processus ne peut connaître l'identité du processus qui lui a envoyé le signal.
- Les signaux sont identifiés par un numéro entier et un nom symbolique décrit dans **signal.h**.
- La commande **kill** \ **-l** donne la liste des signaux (Linux).

Un signal peut être envoyé

1. lors d'une anomalie matérielle (les signaux matériels) ;
2. suite à la frappe d'une combinaison de touches (par ex. CTRL-C pour envoyer le signal SIGINT) ;
3. par un autre processus utilisant la commande kill ou l'appel système kill.

A la réception du signal, trois actions par défaut suivent :

1. ignorer le signal,
2. terminer le programme ou
3. terminer en créant un fichier core.

Un processus peut changer son comportement par défaut lors de la réception d'un signal en déroutant le signal, (indique la fonction à exécuter lors de la réception du signal)

- L'appel système int kill (pid_t *pid*, int *sig*) envoie le signal *sig* au process *pid*.
- La réception d'un signal entraîne un comportement par défaut du process.
Celui-ci peut être modifié en indiquant une autre action à effectuer **(voir signal () ci-après).**
- Un process ne peut envoyer un signal à un autre que si le propriétaire (réel ou effectif) est le même pour les deux process.

Les signaux **SIGKILL** et **SIGSTOP** ne peuvent pas être redéfinis.
Un signal n'est pas reçu lors de son envoie (asynchronisme) ; il existe un délai entre l'envoi et la réception. **SIGINT 2** signal d'interruption (exemple CTRL-C) ; Exemples signal ().

- signal (SIGINT,SIG_IGN) ; :la réception du signal SIGINT devient inopérante ; le signal est ignoré ;
- signal (SIGINT,SIG_DFL) ; :le signal SIGINT retrouve son action par défaut ;
- signal (SIGINT,queFaire) ; :la fonction queFaire() est exécutée à la réception du signal SIGINT. On doit avoir une déclaration de la forme void queFaire(int sig) ;.

1 Introduction

2 Notion de système d'exploitation

3 Programme et processus
- Programme

4 Gestion du process
- Concept du process
  - Définitions
  - État du process
  - Linux/UNIX : Lister des informations sur les process
- Hiérarchie des process
  - UNIX, Linux
- L'ordonnancement
- Communication entre process

5 Les Threads
- Définition
- Utilisation
- Threads POSIX

- **fork** : appel système lourd
- chaque process a son espace mémoire (perte de mémoire)
- chaque process a ses structures systèmes (table des pages, fichiers ouverts...)
- la commutation (passage d'un processus à un autre) lente et lourde à gérer
- communication inter-processus (échange d'informations) généralement lente

Question : *Peut-on avoir un sous-process qui permettrait de lancer une partie du code d'une application sans qu'il soit onéreux* ?
Solution : Les processus légers "threads"

- Un thread appelé *process de poid léger*, est un flux séquentiel de contrôle à l'intérieur d'un programme. Il partage avec les autres threads d'un même process les ressources (mémoire, fichiers, objets, connexion, etc.).
- La programmation par thread permet d'intégrer dans la programmation des applications les bénéfices de la programmation parallèle tout en conservant une partie de la légèreté de la programmation classique (en particulier vis-à-vis de la mémoire).

Exemple

Un premier thread peut lire les données, en même temps un deuxième thread calcule les résultats et en même temps un troisième thread affiche ces résultats. Ces trois threads partagent le même espace d'adressage.

- La création et la destruction des threads sont très rapides par rapport à la création et à la destruction des process,
- L'utilisation des threads permet à une application d'exécuter plusieurs tâches en même temps ce qui accélère son exécution.
- L'utilisation des threads est très intéressante quand la machine est équipé de plusieurs processors, chaque thread peut s'exécuter sur un processor différent..
- Les fonctions de gestion des threads se lancent en même temps que le process principal main (programmation multitâche) ;

Un thread contient :

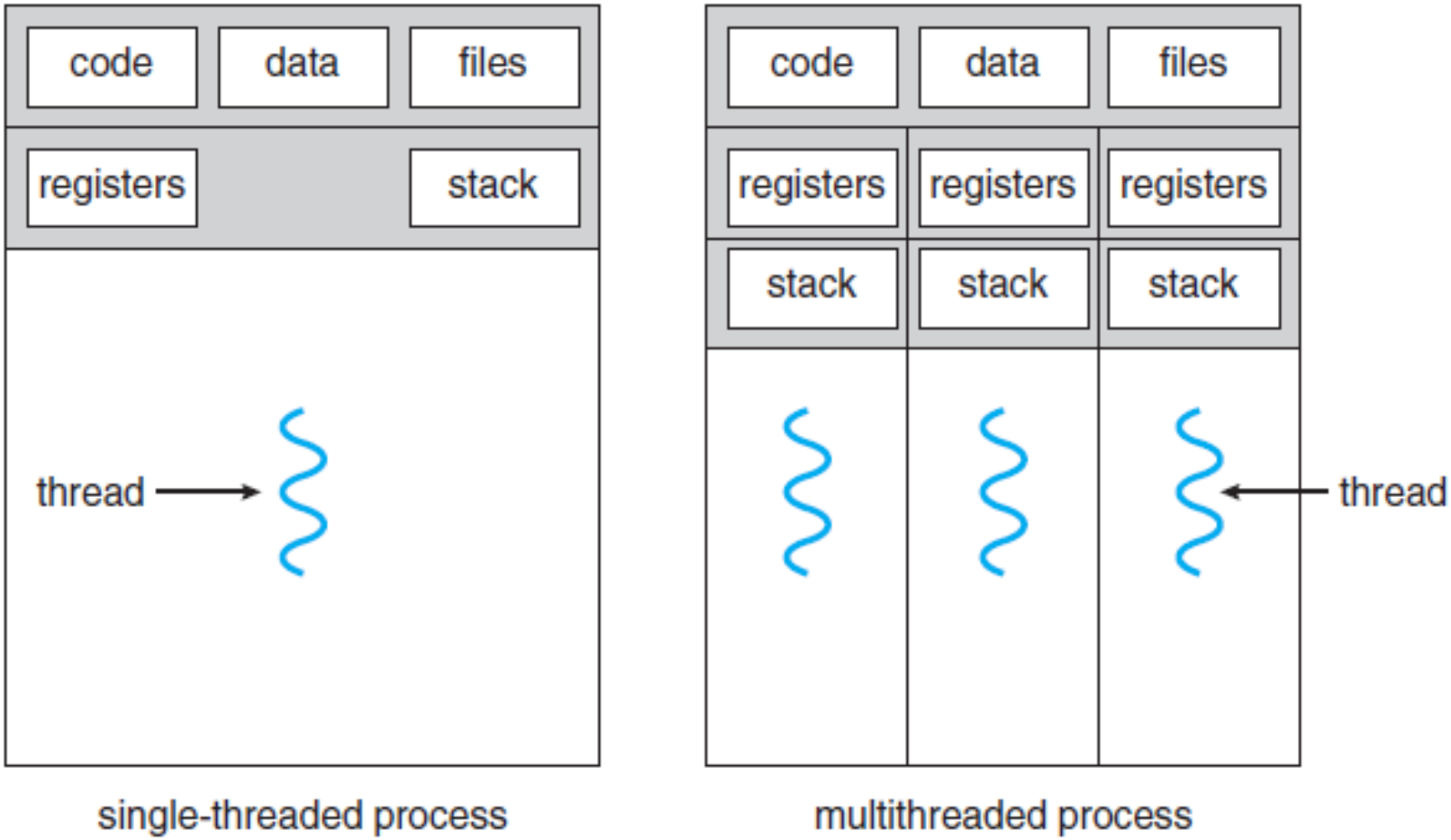
1. compteur d'instructions (priorité d'exécution)
2. ensemble de registres
3. sa propre pile (pour gérer les instructions à exécuter par le thread=

Un thread partage (avec des threads ressemblants)

1. La section de code
2. La section de données
3. Les ressources du OS

Process traditionnels (lourds) : task + thread Threads :

1. **Niveau système (kernel)** : chaque thread est représenté par un processus indépendant, partageant son espace d'adressage avec les autres threads de la même application.
2. **Niveau utilisateur** : l'application n'est constituée que d'un seul processus, et la répartition en différents threads est assurée par une bibliothèque indépendante du noyau.


code
data
files
registers
stack
thread
single-threaded process
code
data
files
registers
registers
registers
stack
stack
stack
thread
multithreaded process

- Le modèle de programmation est plus simple pour des situations complexes.
- La création de threads est plus rapide que celle de processus (facteur 100 sur certains systèmes).
- Il y a une meilleure utilisation des ressources de traitements des machines multiprocesseurs.
- La communication est plus facile entre threads qu'entre processus.
- Un programme multithread répond mieux aux entrée/sortie en leur dédiant des threads.

Inconvénients :

- La commutation de contexte entre threads prend des ressources.
- La programmation multithreads est plus complexe que celle simple thread (synchronisation, débogage).

La création de Thread se fait à l'aide de plusieurs system calls auquel faut passer en paramètre le pointeur de la fonction à exécuter en // La fonction pthread_create() (man 3 pthread_create)

```
#include <pthread.h>
pthread_t thread;

int pthread_create(pthread_t* thread, pthread_attr_t* attr, void*
(*start_routine)(void*), void* arg);
```

Au moment de l'appel du thread, le process courant est cloné partiellement
L'exécution reprend d'un côté à partir de l'endroit où le thread a été appelé et de l'autre dans la fonction.
Vous devez compiler avec une directive particulière avec gcc.
En cas d'erreur, la valeur **EAGAIN** est retournée par la fonction *pthread_create()*
- Pas assez de ressources système pour créer un nouveau thread
- Le nombre maximum de threads définit par la constante PTHREAD_THREADS_MAX est atteint !

les paramètres de la fonction pthread_create :

1. Le paramètre ***thread** est un pointeur de type *pthread_t*
2. Le type **pthread** est un type opaque, sa valeur réelle dépend de l'implémentation ; sous Linux il s'agit en générale du type unsigned long. Ce type correspond à l'identifiant du thread créé ;
3. Chaque thread a son propre identifiant (comme les process Unix)
4. Le paramètre ***attr_t** est un *pointeur* de type *pthread_attr_t*
5. Le type **pthread_attr_t** est aussi un type opaque permettant de définir des attributs spécifiques pour chaque thread. Il faut savoir qu'on peut changer le comportement de la gestion des threads ; exemple, on peut les régler pour qu'ils tournent sur un système temps réel ; en générale on se contente des attributs par défaut et cet argument vaut NULL (valeur par défaut).
6. **La fonction pthread_creat** est un pointeur de fonction. Chaque thread dispose d'une fonction à exécuter, c'est en même temps sa raison d'être ; Il permet de transmettre un pointeur sur la fonction que le thread devra exécuter.
7. Le dernier argument représente un argument que l'on peut passer à la fonction que le thread doit exécuter.

```c
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #define NUM_THREADS     5
5
6 void *PrintHello(void *threadid){
7    long tid;
8    tid = (long)threadid;
9    printf("Hello World! It's me, thread #%ld!\n", tid);
10   pthread_exit(NULL);
11 }
12
13 int main(int argc, char *argv[]){
14 pthread_t threads[NUM_THREADS];
15 int rc;
16 long t;
17 for(t=0;t<NUM_THREADS;t++){
18   printf("In main: creating thread %ld\n", t);
19   rc = pthread_create(&threads[t], NULL, PrintHello, (void *)t);
20   if (rc){
21     printf("ERROR; return code from pthread_create() is %d\n", rc);
22     exit(-1);
23     }
```

```
┌──(kali㉿kali)-[~]
└─$ ./thread
In main: creating thread 0
In main: creating thread 1
In main: creating thread 2
Hello World! It's me, thread #0!
Hello World! It's me, thread #1!
In main: creating thread 3
In main: creating thread 4
Hello World! It's me, thread #2!
Hello World! It's me, thread #3!
Hello World! It's me, thread #4!

┌──(kali㉿kali)-[~]
└─$ ./thread
In main: creating thread 0
In main: creating thread 1
In main: creating thread 2
In main: creating thread 3
In main: creating thread 4
Hello World! It's me, thread #4!
Hello World! It's me, thread #2!
Hello World! It's me, thread #3!
Hello World! It's me, thread #0!
Hello World! It's me, thread #1!

┌──(kali㉿kali)-[~]
└─$
```

pthread_exit

```
void pthread_exit (void * retval) ;
```

La fonction pthread_exit() (man 3 pthread_exit)
Utiliser cette fonction à la place de "exit" afin d'éviter d'arrêter tout le process.
L'appel à une telle fonction à la fin de la fonction main est utile pour éviter que le programme arrête les autres threads en cours.

pthread_cancel

```
int pthread_cancel (pthread_t thread) ;
```

La fonction pour envoyer une demande d'annulation à un thread spécifique

pthread_setcancelstate

```
int pthread_setcancelstate (int state, int * oldstate) ;
```

La fonction pthread_setcancelstate() définit l'état d'annulation du thread appelant à la valeur indiquée par state. L'ancien état d'annulation du thread est renvoyé dans le tampon pointé par oldstate.

IEEE a défini une norme de threads portables (IEEE 1003.1c), Portable Operating System Interface et le X exprime l'héritage UNIX
La norme POSIX définit l'API Pthreads *#include <pthread.h>*
POSIX définit également des mécanismes de synchronisation qui sont, l'exclusion mutuelle (mutex), les sémaphores, et les conditions d'exécution.

Appel	Description
pthread_create	Créer un nouveau thread
pthread_exit	Termine le thread appelant
pthread_join	Attend la fin d'un thread
pthread_yield	Libère l'UC pour laisser un autre thread s'exécuter
pthread_attr_init	Crée et initialise une structure attribut de thread
pthread_attr_destroy	Supprime une structure attribut de thread

le package Pthread contient plus de 60 appels de fonctions.